

CS 426 Examination

Large-Scale System Design and Architecture

Instructions:

- Answer all questions.
 - For Questions 1–4, choose the best option.
 - For Questions 5–8, write brief but precise answers.
1. Which caching strategy provides the strongest consistency guarantees between cache and database?
 - (A) Cache-aside (lazy loading)
 - (B) Write-through caching
 - (C) Write-behind (write-back) caching
 - (D) Read-through caching
 2. In the CAP theorem, which combination is achievable for a distributed database system?
 - (A) Consistency, Availability, and Partition Tolerance simultaneously
 - (B) Consistency and Availability, sacrificing Partition Tolerance
 - (C) Any two of the three properties, but not all three
 - (D) Only one property can be guaranteed in practice
 3. What is the primary purpose of a load balancer in a distributed system architecture?
 - (A) To encrypt network traffic between services
 - (B) To distribute incoming requests across multiple servers for scalability and reliability
 - (C) To cache frequently accessed data
 - (D) To monitor system health metrics
 4. Which database sharding strategy distributes data based on a hash function applied to the shard key?
 - (A) Range-based sharding
 - (B) Hash-based sharding
 - (C) Directory-based sharding
 - (D) Geographic sharding

5. Explain the concept of eventual consistency in distributed systems. What are the trade-offs between strong consistency and eventual consistency, and in what scenarios would you choose eventual consistency?
6. Describe the microservices architecture pattern. What are its advantages over monolithic architecture, and what additional complexities does it introduce in terms of deployment, debugging, and inter-service communication?
7. Explain the purpose and implementation of a distributed message queue (such as Kafka or RabbitMQ). How does it improve system reliability and scalability, and what patterns does it enable?
8. Discuss rate limiting strategies for API services. Compare token bucket and leaky bucket algorithms, and explain how you would implement rate limiting in a distributed system with multiple servers.